

# Kounouz Alafiya

كنوز العافية

## Déploiement gratuit pour la démonstration client — Neon, Render et Vercel

Mettre Kounouz Alafiya en ligne gratuitement, sans carte bancaire et sans serveur à administrer, pour la montrer à un client depuis n'importe quel navigateur : la base de données sur Neon, l'API sur Render, le site sur Vercel. Chaque étape indique où cliquer.

Plateforme de confiance, vérification et traçabilité du miel

Phase 1 · 21/09/2026

## Sommaire

1. En bref
2. Ce qu'il faut savoir avant de commencer
3. Fiche de route à remplir
4. Étape 0 : envoyer la dernière version sur GitHub
5. Étape 1 : créer la base de données (Neon)
6. Étape 2 : charger les données de démonstration
7. Étape 3 : déployer l'API (Render)
8. Étape 4 : déployer le site (Vercel)
9. Étape 5 : relier le site et l'API
10. Étape 6 : vérifier que tout fonctionne
11. Scénario de démonstration (20 minutes)
12. Le jour de la démonstration
13. Après la démonstration
14. Dépannage
15. Passer à la vraie production

## 1. En bref

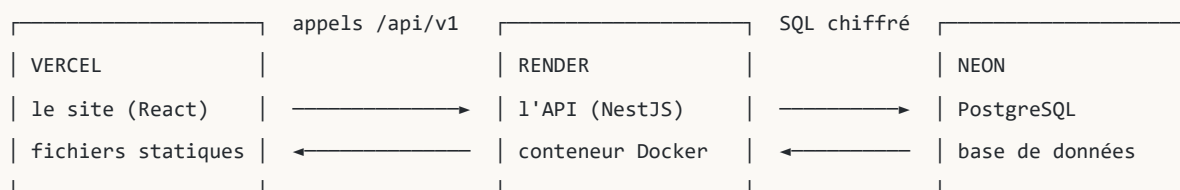
**Ce que vous obtenez** : une adresse du type `https://kounouz-alafiya.vercel.app`, à envoyer à votre client. Elle donne accès aux cinq espaces (producteur, agent terrain, équipe de vérification, administrateur, client) avec un jeu de données complet : 133 demandes de vérification, 7 produits, 1 002 codes QR, 71 commandes, plus de 3 000 scans et une centaine d'alertes anti-contrefaçon.

**Durée** : environ 45 minutes la première fois, dont 10 à 15 minutes d'attente pendant les constructions.

**Coût** : 0. Aucun des trois services ne demande de carte bancaire pour son offre gratuite, à ma connaissance. Si l'un d'eux vous en demande une, vérifiez que vous avez bien choisi l'offre « Free ».

Client (navigateur)

| `https://kounouz-alafiya.vercel.app`



**Ce qui a été vérifié avant la rédaction** : une répétition complète en local a reproduit ce montage : l'API dans une image Docker limitée comme l'offre gratuite de Render (512 Mo de mémoire, 0,1 processeur, port 10000), la base sur un PostgreSQL vide, le site construit avec l'adresse de l'API en dur et servi depuis **une autre origine**. Résultats : les 4 rôles se connectent, le lien direct `/verify/...` fonctionne, les appels entre origines sont acceptés, et toutes les scènes du scénario de la section 11 passent.

**Ce qui n'a pas pu être vérifié** : les écrans de Neon, Render et Vercel eux-mêmes (cela demande vos comptes). Les libellés des boutons sont ceux que ces services affichent en anglais à la date de rédaction ; ils peuvent légèrement changer, l'ordre des étapes reste le même.

## 2. Ce qu'il faut savoir avant de commencer

L'offre gratuite est faite pour une démonstration, pas pour un site en production. Six limites, à connaître pour ne pas être surpris devant le client :

| Limite                                                                           | Conséquence                                                                                                                                                                                                                                                   | Que faire                                                                                                               |
|----------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <b>Render endort l'API après 15 minutes sans visite</b>                          | Le premier accès après une pause prend <b>environ 70 secondes</b> (mesuré avec les limites de l'offre gratuite)                                                                                                                                               | Réveillez l'API 10 minutes avant la démonstration (section 12)                                                          |
| <b>Render efface les fichiers envoyés</b> à chaque endormissement ou mise à jour | Les photos et documents téléversés pendant la démonstration disparaissent ensuite                                                                                                                                                                             | Ne comptez pas dessus ; les données (comptes, demandes, produits) sont, elles, dans Neon et restent                     |
| <b>Neon : 100 « CU-heures » de calcul par mois</b> (0,5 Go de stockage)          | Le calcul de Neon <b>reste actif tant que l'API est éveillée</b> , car l'API interroge la base toutes les 2 secondes. À la taille minimale (0,25 CU), une base active 24 h sur 24 consomme environ 180 CU-heures par mois : le quota serait épuisé vers le 17 | Ne maintenez l'API éveillée que pendant la journée de démonstration (section 12)                                        |
| <b>Vercel Hobby : usage personnel et non commercial</b>                          | Convient à une démonstration temporaire, pas à un site qui vend                                                                                                                                                                                               | Pour la vraie mise en ligne, voir la section 15                                                                         |
| <b>Comptes de démonstration publics</b>                                          | Leurs mots de passe figurent dans la documentation : toute personne qui connaît l'adresse peut se connecter en administrateur                                                                                                                                 | N'envoyez l'adresse qu'à votre client, n'y saisissez aucune donnée réelle, et arrêtez le service après la démonstration |
| <b>Pas de paiement en ligne</b>                                                  | Une commande est enregistrée, le paiement n'existe pas dans l'application                                                                                                                                                                                     | Présentez-le comme le processus de commande ; le paiement reste à définir                                               |

## 3. Fiche de route à remplir

Notez ces valeurs au fur et à mesure : chaque étape en produit une, et la suivante l'utilise.

| Valeur                                                           | Où la trouver                          | Votre valeur |
|------------------------------------------------------------------|----------------------------------------|--------------|
| Adresse de la base Neon ( <code>DATABASE_URL</code> )            | Étape 1                                |              |
| Trois secrets aléatoires                                         | Étape 3 (générés sur votre ordinateur) |              |
| Adresse de l'API Render ( <code>https://...onrender.com</code> ) | Étape 3                                |              |
| Adresse du site Vercel ( <code>https://...vercel.app</code> )    | Étape 4                                |              |

Trois comptes gratuits sont nécessaires : **Neon**, **Render** et **Vercel**. Le plus simple est de s'inscrire avec « Continue with GitHub » sur chacun (vous êtes déjà connecté à GitHub).

## 4. Étape 0 : envoyer la dernière version sur GitHub

Deux fichiers ont été modifiés depuis le dernier envoi :

- `frontend/vercel.json` : **indispensable**. Sans lui, un lien direct comme `/verify/KZ-QR-2026-000001` (celui qu'on scanne sur un pot) donnerait une erreur 404 sur Vercel.
- `backend/prisma/seed.ts` : le chargement des données de démonstration pouvait échouer et laisser la base à moitié vidée. Il est désormais réexécutable à volonté.

Dans le dossier du projet, sur votre ordinateur :

```
git add -A
```

```
git commit -m "Déploiement démo : vercel.json et seed réexécutable"
```

```
git push
```

Vérifiez sur <https://github.com/haythem-djebbi9/kounouz-alafiya> que le fichier `frontend/vercel.json` apparaît.

## 5. Étape 1 : créer la base de données (Neon)

1. Ouvrez <https://neon.com> et cliquez sur **Sign Up** (en haut à droite), puis **Continue with GitHub**. Autorisez l'accès.
2. Neon vous propose de créer un projet. Remplissez :
  - **Project name** : `kounouz-alafiya`
  - **Postgres version** : `16` (la version testée)
  - **Cloud service provider** : `AWS`
  - **Region** : **AWS Europe (Frankfurt)**, la plus proche de la Tunisie parmi les régions proposées. Retenez cette région : vous choisirez la même chez Render.
3. Cliquez sur **Create project**.
4. Sur le tableau de bord du projet, cliquez sur le bouton **Connect** (parfois « Connection Details »). Un panneau affiche l'adresse de connexion. Réglez-le ainsi :
  - **Branch** : `main`
  - **Database** : `neondb`
  - **Role** : `neondb_owner`
  - **Connection pooling** : **décochez** cette case (interrupteur sur « off »). L'adresse ne doit **pas** contenir `-pooler`.
5. Cliquez sur **Show password**, puis sur **Copy snippet** (ou copiez l'adresse). Elle ressemble à :

```
postgresql://neondb_owner:npg_XXXXXXX@ep-quiet-sun-123456.eu-central-1.aws.neon.tech/neondb?
sslmode=require&channel_binding=require
```

6. **Modifiez-la** dans un bloc-notes :

- supprimez `&channel_binding=require` s'il est présent ;
- ajoutez à la fin `&connect_timeout=30&connection_limit=5` .

Vous obtenez l'adresse finale, à noter dans la fiche de route :

```
postgresql://neondb_owner:npg_XXXXXXX@ep-quiet-sun-123456.eu-central-1.aws.neon.tech/neondb?
sslmode=require&connect_timeout=30&connection_limit=5
```

| Paramètre                       | Rôle                                                                                                   |
|---------------------------------|--------------------------------------------------------------------------------------------------------|
| <code>sslmode=require</code>    | Connexion chiffrée, obligatoire chez Neon                                                              |
| <code>connect_timeout=30</code> | Laisse 30 secondes à Neon pour se réveiller après une pause (il s'endort après 5 minutes d'inactivité) |
| <code>connection_limit=5</code> | Évite de saturer la base avec trop de connexions                                                       |

Ce mot de passe donne un accès total à la base. Ne le publiez pas, ne l'écrivez pas dans un fichier du projet.

## 6. Étape 2 : charger les données de démonstration

Cette étape se fait **depuis votre ordinateur** : elle crée les tables dans Neon, puis y met les données de démonstration (les six comptes, les producteurs, les demandes, les produits, les codes QR, les commandes...).

**Attention : le chargement commence par vider entièrement la base visée.** Il ne doit viser que Neon. La commande de contrôle ci-dessous affiche l'adresse visée : lisez-la avant de continuer. Votre base de développement locale n'est jamais concernée si l'adresse affichée contient `neon.tech` .

Ouvrez **PowerShell** et placez-vous dans le dossier de l'API :

```
cd "C:\Users\djebb\Desktop\kounouz affia\backend"
```

Si ce n'est pas déjà fait, installez les dépendances :

```
npm install
```

Indiquez à cette fenêtre l'adresse de Neon (remplacez par **votre** adresse finale de l'étape 1, entre guillemets) :

```
$env:DATABASE_URL = "postgresql://neondb_owner:npg_XXXXXXX@ep-quiet-sun-123456.eu-central-1.aws.neon.tech/neondb?sslmode=require&connect_timeout=30&connection_limit=5"
```

### Contrôle de sécurité :

```
npx prisma migrate status
```

Vous devez lire une ligne de ce type. Si elle affiche `localhost`, **arrêtez-vous** et recommencez la commande précédente :

```
Datasource "db": PostgreSQL database "neondb", schema "public" at "ep-quiet-sun-123456.eu-central-1.aws.neon.tech"
15 migrations found in prisma/migrations
Following migrations have not yet been applied: ...
```

**Si vous lisez `P1001: Can't reach database server` au lieu de la ligne ci-dessus**, ce n'est ni Neon ni votre adresse : votre réseau bloque le port 5432 (celui de PostgreSQL). C'est fréquent avec un **VPN** (Proton VPN, NordVPN...), un pare-feu d'entreprise ou certains opérateurs. **Désactivez le VPN** et relancez la commande. Si l'erreur persiste sans VPN, passez sur un autre réseau (partage de connexion du téléphone) ou consultez le dépannage de la section 14. Attention : `Test-NetConnection` peut afficher « ouvert » alors que le port est bloqué ; seule la commande `migrate status` fait foi.

Créez les tables (quelques secondes) :

```
npx prisma migrate deploy
```

Message attendu : `All migrations have been successfully applied.` Puis chargez les données :

```
npm run prisma:seed
```

Comptez **3 à 8 minutes** : le chargement envoie environ 2 500 requêtes à Neon, une par une, et la durée dépend de votre connexion. Ne fermez pas la fenêtre. La fin est annoncée par la liste des comptes de démonstration :

```
Seed terminé.
  admin@kounouzalafiya.com      (Admin123!)
  ...
```

Retirez l'adresse de la fenêtre pour éviter toute erreur ensuite :

```
Remove-Item Env:DATABASE_URL
```

**Vérification (facultative)** : dans Neon, menu **SQL Editor**, exécutez `select count(*) from users;` : le résultat doit être **45**.

## 7. Étape 3 : déployer l'API (Render)

### 7.1 Préparer les trois secrets

Sur votre ordinateur, dans PowerShell, exécutez **trois fois** cette commande et notez les trois résultats (ils serviront de `JWT_ACCESS_SECRET`, `JWT_REFRESH_SECRET` et `FILES_SIGNING_SECRET`). Les trois valeurs doivent être différentes :

```
node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"
```

### 7.2 Créer le service

1. Ouvrez <https://render.com>, cliquez sur **Get Started**, puis **GitHub**. Autorisez l'accès.
2. Sur le tableau de bord, cliquez sur **+ New** (en haut à droite), puis **Web Service**.
3. Onglet **Git Provider**. Si votre dépôt n'apparaît pas, cliquez sur **Configure account** (ou **Connect GitHub**), choisissez **Only select repositories**, cochez `kounouz-alafiya` et validez. Revenez sur Render, cliquez sur le dépôt `kounouz-alafiya`, puis **Connect**.
4. Remplissez le formulaire :

| Champ          | Valeur                                                  |
|----------------|---------------------------------------------------------|
| Name           | <code>kounouz-api</code>                                |
| Language       | <code>Docker</code>                                     |
| Branch         | <code>master</code>                                     |
| Region         | <b>Frankfurt (EU Central)</b> , la même région que Neon |
| Root Directory | <code>backend</code>                                    |
| Instance Type  | <b>Free</b>                                             |

Laissez **Dockerfile Path** à `./Dockerfile`. Ne renseignez **pas** de « Start Command » ni de « Build Command » : le Dockerfile s'en charge, et il applique aussi les migrations au démarrage.

5. Descendez jusqu'à **Environment Variables**. Cliquez sur **Add from .env**, collez le bloc ci-dessous **après avoir remplacé les valeurs**, puis validez :

```
DATABASE_URL=COLLEZ_ICI_L_ADRESSE_NEON_FINALE
JWT_ACCESS_SECRET=PREMIER_SECRET
JWT_REFRESH_SECRET=DEUXIEME_SECRET
FILES_SIGNING_SECRET=TROISIEME_SECRET
PUBLIC_APP_URL=https://kounouz-alafiya.vercel.app
CORS_ORIGINS=https://kounouz-alafiya.vercel.app
TRUST_PROXY=1
THROTTLE_LIMIT=600
KOUNOUZ_COMMISSION_RATE=0.2
```

| Variable                                                            | Rôle                                                                                                                     |
|---------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <code>DATABASE_URL</code>                                           | L'adresse finale de l'étape 1                                                                                            |
| <code>JWT_ACCESS_SECRET</code> ,<br><code>JWT_REFRESH_SECRET</code> | Signature des sessions. L'API refuse de démarrer sans eux                                                                |
| <code>FILES_SIGNING_SECRET</code>                                   | Signature des liens vers les fichiers privés                                                                             |
| <code>PUBLIC_APP_URL</code>                                         | Adresse du site : elle est <b>encodée dans les codes QR</b> . Provisoire pour l'instant, corrigée à l'étape 5            |
| <code>CORS_ORIGINS</code>                                           | Seule adresse autorisée à appeler l'API depuis un navigateur. Provisoire, corrigée à l'étape 5                           |
| <code>TRUST_PROXY</code>                                            | L'API lit ainsi l'IP réelle des visiteurs derrière le relais de Render                                                   |
| <code>THROTTLE_LIMIT</code>                                         | Nombre de requêtes autorisées par minute et par visiteur (120 par défaut ; relevé pour ne jamais gêner la démonstration) |

Ne définissez **pas** `PORT` : Render en fournit un (10000) que l'API utilise automatiquement.

6. Cliquez sur **Advanced**, puis :

- **Health Check Path** : `/health`
- **Auto-Deploy** : `On Commit` (chaque envoi sur GitHub redéploiera l'API)

7. Cliquez sur **Deploy Web Service** (en bas de page).

### 7.3 Attendre et contrôler

Render ouvre la page **Logs**. La construction de l'image dure **5 à 10 minutes** (`npm ci`, génération de Prisma, compilation). Elle se termine par une ligne :

```
Nest application successfully started
```

et l'étiquette **Live** (en vert) apparaît en haut de la page. Sous le nom du service se trouve son adresse, du type `https://kounouz-api-xxxx.onrender.com`. **Notez-la dans la fiche de route.**

Ouvrez dans le navigateur (remplacez par votre adresse) :



| Adresse                                                                    | Résultat attendu                                       |
|----------------------------------------------------------------------------|--------------------------------------------------------|
| <code>https://kounouz-api-xxxx.onrender.com/health</code>                  | <code>{"status":"ok", ...}</code>                      |
| <code>https://kounouz-api-xxxx.onrender.com/ready</code>                   | <code>"database":"up"</code> : l'API parle bien à Neon |
| <code>https://kounouz-api-xxxx.onrender.com/api/v1/products/catalog</code> | Une liste JSON de 4 produits                           |

Si `/ready` indique `"database":"down"`, l'adresse Neon est incorrecte (section 14).

## 8. Étape 4 : déployer le site (Vercel)

- Ouvrez <https://vercel.com/signup>, choisissez le plan **Hobby**, puis **Continue with GitHub**. Autorisez l'accès.
- Sur le tableau de bord, cliquez sur **Add New...**, puis **Project**.
- Dans **Import Git Repository**, cliquez sur **Import** en face de `kounouz-alafiya`. Si le dépôt n'apparaît pas, cliquez sur **Adjust GitHub App Permissions**, autorisez le dépôt, puis revenez.
- Sur l'écran **Configure Project** :
  - Project Name** : `kounouz-alafiya` (l'adresse du site en découle)
  - Framework Preset** : `Vite` (détection automatique)
  - Root Directory** : cliquez sur **Edit**, choisissez le dossier `frontend`, puis **Continue**.
  - Laissez **Build and Output Settings** tels quels : le fichier `vercel.json` les fixe (`npm ci`, `npm run build`, dossier `dist`).
- Déroulez **Environment Variables** et ajoutez **une** variable :

| Name                      | Value                                                     |
|---------------------------|-----------------------------------------------------------|
| <code>VITE_API_URL</code> | <code>https://kounouz-api-xxxx.onrender.com/api/v1</code> |

Remplacez par **l'adresse de votre API** (étape 3) suivie de `/api/v1`, sans barre oblique finale. Cliquez sur **Add**.

Cette variable est intégrée au site **au moment de sa construction**. Si vous la modifiez plus tard, il faut relancer un déploiement (section 14). Si elle est absente, le site cherche l'API sur `localhost` et n'affiche aucun produit.

- Cliquez sur **Deploy**. La construction dure 1 à 3 minutes.
- Vercel affiche « Congratulations! » avec une vignette du site. Cliquez sur **Continue to Dashboard**. L'adresse du site se trouve sous **Domains**, par exemple `kounouz-alafiya.vercel.app` (si le nom est déjà pris, Vercel ajoute un suffixe : utilisez l'adresse affichée). **Notez-la dans la fiche de route**.

## 9. Étape 5 : relier le site et l'API

L'API doit connaître l'adresse exacte du site, pour deux raisons : autoriser ses appels (sinon le navigateur les bloque) et écrire la bonne adresse dans les codes QR.

1. Sur **Render**, cliquez sur le service **kounouz-api**, puis **Environment** dans le menu de gauche.
2. Cliquez sur **Edit** (ou le crayon) et corrigez les deux variables avec **l'adresse exacte du site Vercel** :
  - `PUBLIC_APP_URL` = `https://kounouz-alafiya.vercel.app`
  - `CORS_ORIGINS` = `https://kounouz-alafiya.vercel.app`

Règles : commencez par `https://`, **pas de barre oblique à la fin**, pas d'espace. Une lettre de différence avec l'adresse du site et le navigateur refuse les appels.

3. Cliquez sur **Save, rebuild, and deploy** (ou **Save and deploy**). L'API redémarre en 2 à 3 minutes. Attendez l'étiquette **Live**.

## 10. Étape 6 : vérifier que tout fonctionne

Ouvrez l'adresse du site Vercel dans le navigateur. Si l'API dormait, la première ouverture peut prendre environ une minute.

| Vérification                | Comment                                                            | Résultat attendu                                                                        |
|-----------------------------|--------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| Le site charge les produits | Page d'accueil, section « Produits vérifiés »                      | 4 produits avec des prix en dinars ( <code>89,900</code> ... د.ت, <code>31</code> د.ت ) |
| Le lien direct fonctionne   | Ouvrir <code>https://...vercel.app/verify/KZ-QR-2026-000001</code> | « Produit vérifié », avec l'origine (Le Kef) et le lot <code>KZ-BAT-0001</code>         |
| Un produit suspendu         | Ouvrir <code>.../verify/KZ-QR-2026-000002</code>                   | « Produit temporairement suspendu »                                                     |
| La connexion                | <code>/connexion</code> , compte administrateur (section 11)       | Arrivée sur le tableau de bord admin                                                    |
| Aucune erreur cachée        | Touche <b>F12</b> , onglet <b>Console</b>                          | Aucun message rouge « CORS » ou « blocked »                                             |

Changez la langue avec l'icône  en haut de page : l'application existe en arabe (par défaut), français et anglais.

Si un point échoue, la section 14 donne la cause la plus probable.

## 11. Scénario de démonstration (20 minutes)

Chaque scène a été testée sur la copie locale du montage. L'ordre suit le parcours réel d'un miel, du producteur au client.

## Comptes

| Espace                 | E-mail                          | Mot de passe |
|------------------------|---------------------------------|--------------|
| Administrateur         | admin@kounouzalafiya.com        | Admin123!    |
| Équipe de vérification | verification@kounouzalafiya.com | Verif123!    |
| Agent terrain          | agent@kounouzalafiya.com        | Agent123!    |
| Producteur             | producteur@kounouzalafiya.com   | Prod123!     |
| Client                 | client@kounouzalafiya.com       | Client123!   |

Une seule session à la fois par navigateur : pour changer d'espace, cliquez sur le nom en haut à droite puis **Déconnexion**, ou ouvrez une fenêtre de navigation privée par espace.

### Scène 1 : le visiteur (3 minutes)

1. Page d'accueil : faites défiler le **parcours de confiance** (les 6 étapes) et les produits en dinars.
2. Cliquez sur **Vérifier un produit**, saisissez **KZ-QR-2026-000001** : le produit est **vérifié**, avec l'origine, le lot et les résultats d'analyse (section dépliant).
3. Saisissez **KZ-QR-2026-000002** : **produit suspendu**.
4. Saisissez **KZ-QR-FAUX-1** : **introuvable**. Ce scan est enregistré et déclenche une alerte anti-contrefaçon : vous la retrouverez à la scène 6.

### Scène 2 : le producteur (4 minutes)

Connectez-vous avec **producteur@kounouzalafiya.com**.

1. **Tableau de bord** : vue d'ensemble.
2. **Demandes de vérification** : ouvrez **VR-2026-090** (Miel de Sauge, « Complément demandé »). Lisez la question de Kounouz, écrivez une réponse et cliquez sur **Envoyer ma réponse**.
3. **Nouvelle demande de vérification** : parcourez l'assistant en 4 étapes et soumettez. À la première étape, choisissez « **Kounouz vient sur mon exploitation** » : c'est ce choix qui permet d'assigner un agent à la scène 3. Les deux documents obligatoires (CIN, attestation d'immatriculation) sont déjà validés, donc la soumission passe.
4. **Ventes & gains** : les ventes et la commission de 20 %.

### Scène 3 : l'équipe de vérification (6 minutes)

Connectez-vous avec **verification@kounouzalafiya.com**.

1. **Revue des demandes** : ouvrez la demande que vous venez de créer, cliquez sur **Prendre en charge**, puis **Accepter**. Dans **Visite de collecte**, cliquez sur **Assigner un agent**.
2. **Décision de vérification** : la file contient 7 dossiers en attente (des analyses conformes, clôturées). Cliquez sur **Examiner** sur l'un d'eux et notez le code de son échantillon (du type **SM-2026-058**).
3. Dans **Pièces du dossier**, une seule pièce est en défaut : **l'échantillon de référence**. L'option **Approuver** est **grisée** : c'est la règle de sécurité du produit, aucun miel ne peut être déclaré vérifié sans sa portion de référence conservée. Montrez-le au client.

4. **Gestion des échantillons** : onglet **Analyse terminée**, recherchez le code de l'échantillon et ouvrez-le. Onglet **Référence**, remplissez le lieu de conservation (par exemple « Chambre froide B, étagère 3 »), les conditions (« 4 °C, à l'abri de la lumière »), la durée « 24 mois », puis **Enregistrer l'échantillon de référence**.
5. Retournez dans **Décision de vérification**, rouvrez le dossier : la pièce est passée au vert et **Approuver** est disponible. Choisissez-le, puis **Confirmer la décision**. Elle est définitive.
6. **Lots vérifiés** : **Créer un lot vérifié**. Le lot reçoit un code **KZ-BAT-2026-...**.

#### Scène 4 : la mise en marché en direct (3 minutes)

Toujours avec l'équipe de vérification.

1. Menu **Produits** : la file **Lots emballés** contient les lots devenus produits. Ouvrez le lot **KZ-BAT-2026-024** : c'est le produit **Kounouz Miel de Sauge**, en brouillon, avec ses 200 codes QR déjà générés.
2. Cliquez sur **Mettre en marché**.
3. Dans un autre onglet, rechargez la page d'accueil du site : le produit **apparaît dans la boutique**, immédiatement (le catalogue passe de 4 à 5 produits).

Pour montrer aussi le début de cette chaîne, ouvrez **Emballage** : plusieurs lots vérifiés y attendent leur emballage ( **KZ-BAT-2026-003** à **006** ), puis le menu **QR codes** pour la génération des codes.

#### Scène 5 : l'agent terrain, sur téléphone (2 minutes)

Ouvrez le site sur un téléphone (ou réduisez la fenêtre du navigateur) et connectez-vous avec **agent@kounouzalafiya.com**.

1. **Mes missions** : 5 missions du jour. Ouvrez **CO-2026-001** (en cours).
2. **Collecte d'échantillon** : montrez l'assistant en 5 étapes (mission, collecte avec GPS, scellé, confirmation).
3. **Chaîne de possession** : le trajet d'un échantillon jusqu'au laboratoire.

#### Scène 6 : l'administrateur (4 minutes)

Connectez-vous avec **admin@kounouzalafiya.com**.

1. **Tableau de bord** : producteurs, laboratoires, produits, lots, scans.
2. **Analyses** → **Alertes anti-contrefaçon** : une dizaine d'alertes ouvertes, dont celle du faux code de la scène 1 (type « QR invalide », gravité haute). Ouvrez-en une : le code, l'appareil, l'IP, l'historique. Changez son statut et ajoutez une note d'enquête.
3. **Analyses** → **Analyse des scans QR** : scans dans le temps, par pays et par appareil.
4. **Journal d'audit** : chaque action des scènes précédentes y est inscrite (qui, quoi, quand).

#### Scène 7 : le client achète (2 minutes)

1. Dans le site, ajoutez **Miel de Sedra Premium 500g** au panier, puis **Commander**. Renseignez nom, téléphone, ville et adresse. Le total s'affiche en dinars (2 pots à 89,900 DT + 25 DT de livraison = **204,800 DT**).
2. Reconnectez-vous en administrateur : **Ventes** → **Commandes** : la commande est **en attente**. Cliquez sur **Confirmer**, puis **Marquer expédiée**.

## 12. Le jour de la démonstration

### Réveiller l'API (10 minutes avant)

Une API endormie met environ 70 secondes à répondre. Ouvrez, dans un onglet, l'adresse :

```
https://kounouz-api-xxxx.onrender.com/ready
```

Attendez le message `{"status": "ready", ...}`. Cela réveille l'API **et** Neon. Ouvrez ensuite le site : il répond immédiatement.

### Empêcher l'API de se rendormir pendant la démonstration

Sans visite pendant 15 minutes, l'API s'endort à nouveau. Deux façons de l'éviter :

- **Simple** : gardez un onglet du site ouvert et cliquez de temps en temps.
- **Automatique** : sur <https://uptimerobot.com> (gratuit), créez un moniteur **HTTP(s)** sur <https://kounouz-api-xxxx.onrender.com/health> avec un intervalle de 5 minutes. **Mettez-le en pause dès la démonstration terminée** : tant qu'il tourne, Neon reste actif et consomme son quota mensuel de 100 CU-heures (section 2).

### Données fraîches

Les missions de l'agent sont datées du jour du chargement des données. Pour que l'onglet **Aujourd'hui** de l'agent soit rempli, **rechargez les données de démonstration le matin même** (section 13). Comptez 3 à 8 minutes.

### Si le client vous suit depuis son propre navigateur

Il peut utiliser les mêmes comptes. Une seule personne à la fois sur un même compte évite de mélanger les actions dans le journal d'audit.

## 13. Après la démonstration

### Mettre à jour l'application

Chaque envoi sur GitHub redéploie automatiquement les deux services :

```
git add -A && git commit -m "Description de la modification" && git push
```

Render reconstruit l'API (5 à 10 minutes) et Vercel le site (1 à 3 minutes). Si vous modifiez `VITE_API_URL` dans Vercel, relancez le site à la main : **Deployments** → menu ⋮ du dernier déploiement → **Redeploy**.

### Remettre les données de démonstration à zéro

Recommencez l'étape 2 (section 6) : `migrate status` pour contrôler la cible, puis `npm run prisma:seed`. Les données actuelles sont effacées et remplacées par le jeu d'origine. Les sessions ouvertes sont invalidées : les utilisateurs doivent se reconnecter.

## Arrêter la démonstration

À faire si le client n'a plus besoin de la démonstration (les comptes de démonstration sont publics) :

| Service | Où cliquer                                                                                                                       |
|---------|----------------------------------------------------------------------------------------------------------------------------------|
| Render  | Service <b>kounouz-api</b> → <b>Settings</b> → <b>Suspend Web Service</b> (réversible)                                           |
| Vercel  | Projet → <b>Settings</b> → <b>General</b> → tout en bas <b>Delete Project</b> , ou laissez-le : sans API, il n'affiche plus rien |
| Neon    | Projet → <b>Settings</b> → <b>Delete project</b> (irréversible) ; sinon la base s'endort seule et ne consomme rien               |

Pour la reprendre plus tard : **Resume** chez Render, puis réveillez l'API (section 12).

---

## 14. Dépannage

| Symptôme                                                                                                                      | Cause probable                                                                                                   | Solution                                                                                                                                                                                            |
|-------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Le site s'affiche mais <b>sans produits</b> , ou message « erreur réseau »                                                    | <code>VITE_API_URL</code> absente, fausse, ou site construit avant de l'avoir renseignée                         | Vercel → <b>Settings</b> → <b>Environment Variables</b> : vérifiez la valeur (adresse de l'API + <code>/api/v1</code> ), puis <b>Redeploy</b>                                                       |
| Même symptôme, mais l'API répond quand on ouvre son adresse                                                                   | Erreur <b>CORS</b> : <code>CORS_ORIGINS</code> ne correspond pas exactement à l'adresse du site                  | F12 → <b>Console</b> : un message « blocked by CORS policy » le confirme. Corrigez <code>CORS_ORIGINS</code> chez Render ( <code>https://</code> , sans <code>/ final</code> ), et redéployez l'API |
| Le site met plus d'une minute à répondre                                                                                      | L'API dormait                                                                                                    | Attendez : c'est normal une fois. Réveillez-la avant la démonstration (section 12)                                                                                                                  |
| <b>404</b> sur <code>/verify/...</code> ou après un rafraîchissement                                                          | <code>frontend/vercel.json</code> absent de GitHub                                                               | Faites l'étape 0, puis attendez le déploiement automatique                                                                                                                                          |
| Render : <b>Build failed</b>                                                                                                  | Erreur pendant la construction                                                                                   | Onglet <b>Logs</b> : lisez les dernières lignes. Vérifiez <b>Root Directory</b> = <code>backend</code> et <b>Language</b> = <code>Docker</code>                                                     |
| Render : le déploiement échoue au démarrage avec <code>JWT_ACCESS_SECRET</code>                                               | Un secret est manquant                                                                                           | Ajoutez les variables de l'étape 3 (point 5)                                                                                                                                                        |
| Sur <b>votre ordinateur</b> , <code>P1001: Can't reach database server</code> alors que l'adresse est juste                   | Le réseau bloque le port 5432 (VPN actif, pare-feu, opérateur). Le port s'ouvre mais le serveur ne répond jamais | Désactivez le VPN ; sinon changez de réseau (partage de connexion du téléphone). Vérifiez avec <code>npx prisma migrate status</code>                                                               |
| Sur <b>Render</b> , <code>/ready</code> indique <code>"database": "down"</code> , ou <code>Can't reach database server</code> | Adresse Neon mal copiée, ou mot de passe faux                                                                    | Recopiez l'adresse depuis Neon. Vérifiez qu'elle se termine par <code>? sslmode=require&amp;connect_timeout=30&amp;connection_limit=5</code> et qu'elle ne contient pas <code>-pooler</code>        |
| <code>Invalid credentials</code> à la connexion                                                                               | Les données n'ont pas été chargées dans <b>Neon</b>                                                              | Refaites l'étape 2 en contrôlant que l'adresse affichée contient <code>neon.tech</code>                                                                                                             |
| Le chargement ( <code>prisma:seed</code> ) échoue avec « Foreign key constraint »                                             | Ancienne version du seed                                                                                         | Faites l'étape 0 puis récupérez la dernière version du dépôt                                                                                                                                        |
| Erreur <code>429 Too Many Requests</code>                                                                                     | Limite de requêtes atteinte                                                                                      | Relevez <code>THROTTLE_LIMIT</code> chez Render (par exemple 1000)                                                                                                                                  |

| Symptôme                                                             | Cause probable                                                                    | Solution                                                                                                                               |
|----------------------------------------------------------------------|-----------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Neon : <code>compute time quota exceeded</code>                      | Les 100 CU-heures du mois sont consommées (API maintenue éveillée trop longtemps) | Attendez le mois suivant, ou passez à l'offre payante de Neon ; désormais, ne maintenez l'API éveillée que le jour de la démonstration |
| Les photos ou documents envoyés pendant la démonstration ont disparu | Le disque de Render est effacé à chaque redémarrage                               | Normal en offre gratuite. Pour les conserver, il faut un vrai serveur (section 15)                                                     |
| Les codes QR renvoient vers une mauvaise adresse                     | <code>PUBLIC_APP_URL</code> incorrecte                                            | Corrigez-la chez Render. Les images de QR se recalculent à chaque affichage : elles se corrigent seules                                |

## 15. Passer à la vraie production

Cette démonstration n'est pas adaptée à la mise en service : API endormie, fichiers effacés, comptes publics, offre Vercel non commerciale, absence de sauvegardes. Pour la vraie mise en ligne, suivez `GUIDE_DEPLOIEMENT_VPS.md` : un serveur Hostinger, un nom de domaine, le HTTPS, des sauvegardes quotidiennes et des fichiers conservés. Les deux guides ne se contredisent pas : les corrections de sécurité de ce dépôt (adresse IP des visiteurs, envoi de fichiers, volumes) servent aux deux.

Le jour de la mise en production, **ne chargez pas les données de démonstration** : créez votre premier administrateur avec le script prévu (section 11 du guide VPS).